INF-239 Bases de Datos
Tarea 3: MongoDB SaludUSM

Profesores: Ricardo Salas (ricardo.salas@usm.cl)
Jose Luis Marti (joseluis.marti@usm.cl)
Ayudante Coordinador: Gonzalo Alarcón (gonzalo.alarconc@usm.cl)

Octubre, 2026

1. Objetivo

Investigar y aplicar los conceptos básicos del gestor de bases de datos NoSQL MongoDB, utilizando Docker para desplegar los servicios bajo una arquitectura replicada, MongoDB Query Language (MQL) y Python, implementando el sistema de gestión clínica de la red de salud SaludUSM desarrollado en la Tarea 2. Además, se busca que los estudiantes diferencien teóricamente las bases de datos relacionales (SQL) de las no relacionales (NoSQL), a partir de su propia implementación en MySQL y en MongoDB.

2. Especificaciones y reglas

El desarrollo de esta tarea debe cumplir las siguientes especificaciones, de lo contrario existirá un descuento en la nota final:

- Se deben ejecutar todos los procedimientos necesarios para implementar la base de datos MongoDB sobre Docker, bajo una arquitectura distribuida maestro-esclavo (*replica set*), utilizando Python (biblioteca PyMongo) como lenguaje de programación para realizar todas las operaciones que se requieran en esta tarea.
- La tarea debe ser entregada el **viernes 06 de Noviembre hasta las 23:59 horas** vía AULA, y su respectiva defensa es obligatoria.

Sobre la entrega

La tarea debe ser entregada como un archivo comprimido de la forma `T3-ROL1-ROL2.zip` y debe contener los siguientes archivos:

1. Archivo `deploy-rol1-rol2.yml` utilizado para desplegar los servicios sobre Docker con las especificaciones de arquitectura entregadas.
2. Archivo Jupyter Notebook o Google Colab `pymongo-rol1-rol2.ipynb`, el cual debe venir **ejecutado** con todas las evidencias de los resultados documentadas en el mismo notebook, considerando una sección por cada requerimiento solicitado en la tarea.
3. **Carpeta JSON:** un archivo `.json` por cada colección de la base de datos (nombrado como la colección), que contenga los datos finales y actualizados que den cuenta de las operaciones realizadas según los requerimientos de esta tarea.
4. **Carpeta PDF:** un archivo `informe-rol1-rol2.pdf` que contenga el diseño del modelo documental (sección 4.1) y el análisis comparativo (sección 4.5).
5. **Carpeta BD:** el script `.sql` de MySQL utilizado como origen de los datos (puede ser el de la Tarea 2, corregido si corresponde) y los scripts `.py` de migración, si se usaron fuera del notebook.

6. Se debe incluir un archivo **README** (`.txt` o `.md`), el cual debe contener nombres, rol de los alumnos y las instrucciones para la correcta ejecución del programa. Ante cualquier duda sobre algo que no aparezca especificado en la tarea y no afecte las reglas de esta, puede asumir lo que estime conveniente, pero debe ser especificado en el README. En caso de asumir situaciones y no especificarlas, habrá descuento.

7. Esta tarea debe ser defendida para obtener la nota. Para esto se habilitará un documento donde podrán inscribir cuándo realizar su defensa según los horarios disponibles del ayudante que les revisará. Quienes inscriban un horario y no asistan tendrán 0 como nota de esta entrega.

8. **Las consultas se deberán hacer mediante el foro de Aula, el cual se encontrará en la sección Tareas, o mediante el foro "Tarea 3" del Discord del curso.**

Reglas

- **Debe** realizarse en parejas, no se aceptarán tareas individuales.
- Las copias serán evaluadas con nota 0 y se informará a las respectivas autoridades.
- En el foro de búsqueda de pareja podrán buscar pareja quienes no tengan; esto es exclusiva responsabilidad del estudiante.
- En caso de problemas con su pareja deberán contactar al profesor explicando su situación (hacerlo con anticipación). Solo un alumno debe realizar la entrega.
- En caso de que falle la ejecución de algún comando, no se asignará puntaje a éste. Existe la posibilidad de que a su defensa asista su profesor y realice preguntas.
- Es responsabilidad del estudiante inscribir un horario de defensa y estar presente en la fecha y hora elegida.
- Cada grupo tendrá un horario definido para su defensa; en caso de atraso contarán con un tiempo menor para presentar su trabajo.
- Durante la defensa se les preguntará sobre el código entregado (Python y MQL) y sobre las decisiones de diseño del modelo documental, debiendo responder correctamente para optar al puntaje completo.
- La información respecto a la defensa será eventualmente publicada en Aula, esto considera el detalle sobre los descuentos. Es obligación del estudiante estar atento a esta información y cumplir con lo establecido allí.
- Las consultas se responderán hasta 72 horas antes de la entrega.

3. Descripción del problema

Tras la puesta en marcha de la plataforma web desarrollada en la Tarea 2, la red SaludUSM ha seguido creciendo en número de centros y pacientes. La dirección ha detectado dos necesidades: por un lado, el agendamiento de horas no puede detenerse si un servidor falla; por otro, el equipo clínico solicita que la ficha de atención pueda incorporar nueva información (por ejemplo, signos vitales o alergias) sin requerir cambios mayores en la estructura de la base de datos.

Por esta razón, la dirección está evaluando migrar parte del sistema a una base de datos NoSQL orientada a documentos. Antes de tomar una decisión, le solicita implementar el **mismo sistema de gestión clínica** de la Tarea 2 (centros médicos, pacientes, médicos, especialidades, citas, atenciones, diagnósticos y recetas) en **MongoDB**, desplegado en un cluster replicado, y entregar un **informe técnico comparativo** que permita decidir qué tecnología conviene utilizar.

La base de datos debe llamarse `salud_usm`. La cantidad, el nombre y la estructura de las colecciones son parte del diseño que deben realizar y justificar. Las reglas de negocio vigentes en la Tarea 2 se mantienen y deben seguir cumpliéndose en la implementación documental.

4. Requisitos

4.1. Diseño del modelo documental

1. Diseñar las colecciones de `salud_usm` a partir del modelo normalizado de la Tarea 2, considerando como patrones de acceso principales las vistas de usuario de la Tarea 2 (*Comprobante de Reserva de Hora* y *Ficha de Atención*).

2. Decidir qué información se guarda **dentro de un mismo documento** (documentos anidados; por ejemplo, los diagnósticos y recetas guardados como arreglos dentro del documento de la atención) y qué información se guarda en **colecciones separadas** relacionadas mediante un identificador (de forma similar a una clave foránea). Esta decisión debe justificarse para cada relación del modelo: cita–atención, atención–diagnóstico, atención–receta, médico–especialidad, médico–centro y paciente–cita.

3. Los catálogos (previsión, especialidad y estado de cita) deben mantenerse como colecciones de catálogo, de modo que sus valores no se ingresen como texto libre.

4. El informe debe incluir un diagrama o descripción de las colecciones y un documento de ejemplo de cada una. A modo referencial, la Figura 1 muestra el formato esperado para un documento; el diseño de las demás colecciones es parte de la evaluación.

```
{
  "_id": "CM01",
  "nombre": "Centro SaludUSM Providencia",
  "comuna": "Providencia",
  "region": "Metropolitana"
}
```

Figura 1: Ejemplo referencial de un documento de centro médico

4.2. Migración y poblamiento

1. Poblar `salud_usm` mediante código Python a partir de los datos de la base MySQL de la Tarea 2 (exportando a CSV/JSON o conectándose directamente a MySQL), respetando como mínimo los siguientes volúmenes: 4 centros en al menos 2 regiones, 8 especialidades, 3 previsiones, 40 pacientes, 15 médicos, 120 citas, 60 atenciones con sus diagnósticos y 80 recetas.

2. Mostrar como evidencia la cantidad de documentos por colección (`count_documents`) junto a los `SELECT COUNT(*)` equivalentes en MySQL, verificando que no se pierde información en la migración.

3. Los datos deben permitir que las cinco consultas de la sección 4.3 retornen resultados no vacíos, y deben existir citas vencidas en estado *Reservada* o *Confirmada* para validar la sección 4.4.

4.3. Consultas

Implementar en Python, utilizando `find()` o el *aggregation pipeline* (`aggregate()`), las siguientes consultas. Los filtros, cruces, agrupaciones y ordenamientos deben resolverse en MongoDB; **no está permitido** procesar los resultados con lógica de Python para obtener la respuesta. Para cada consulta se debe mostrar el código, el resultado y una breve explicación, junto a la consulta SQL equivalente sobre la base MySQL de la Tarea 2.

a. **Agenda diaria de un médico.** Dado el RUT de un médico y una fecha, devolver sus citas de ese día ordenadas por hora, mostrando hora, nombre del paciente, previsión, centro, especialidad y estado.

b. **Historial clínico de un paciente.** Dado el RUT de un paciente, devolver todas sus atenciones ordenadas de la más reciente a la más antigua, mostrando fecha, centro, especialidad, nombre del médico, motivo de consulta, diagnósticos (código CIE-10 y descripción) y recetas (medicamento, dosis y días de tratamiento).

c. **Tasa de inasistencia por centro.** Para cada centro médico, mostrar el total de citas, la cantidad de citas en estado *No Asistió* y el porcentaje que representan (redondeado a 2 decimales), listando únicamente los centros cuyo porcentaje sea mayor a 15 % y ordenando de mayor a menor porcentaje.

d. **Ranking de medicamentos por especialidad.** Dada una especialidad (por ejemplo *Pediatría*), mostrar los 5 medicamentos más recetados en las atenciones de esa especialidad junto a la cantidad de recetas de cada uno. Especifique en el README el criterio de desempate.

e. **Pacientes multi-especialidad.** Obtener los pacientes que han sido atendidos (citas en estado *Atendida*) en 3 o más especialidades distintas, mostrando su nombre, la cantidad de especialidades y el listado de ellas, ordenados de mayor a menor cantidad.

4.4. Replicación

1. Implementar la arquitectura de cluster con un *replica set* de 3 nodos: una de las máquinas será el primario y las otras dos serán secundarios. Esta configuración deberá estar operativa en su máquina local, donde se levantarán las 3 instancias de Docker con los servicios de MongoDB y la redundancia de la base de datos `salud_usm`. Se debe mostrar desde Python el estado del *replica set* identificando el rol de cada nodo.

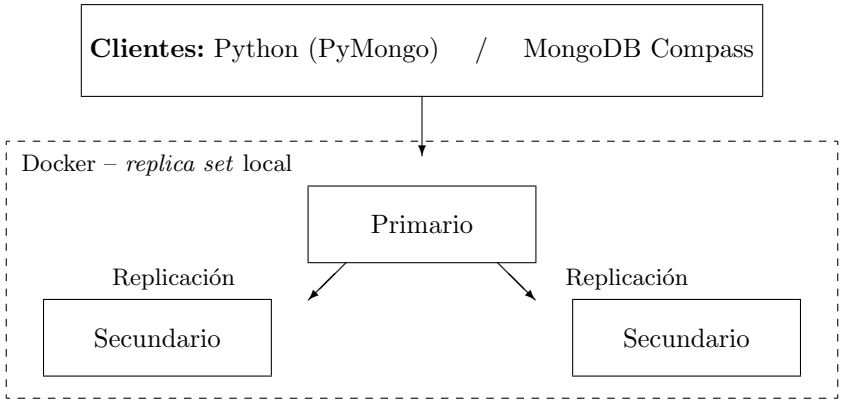


Figura 2: Arquitectura general

2. **Consistencia de los datos.** Realizando las operaciones desde Python sobre el primario (utilizando *write concern* `majority`), se debe demostrar que los datos permanecen consistentes en los 3 nodos después de cada una de las siguientes operaciones:

   - Inserción de una nueva cita en estado *Reservada*.
   - Actualización de las citas vencidas (estado *Reservada* o *Confirmada* con fecha anterior a la actual) a estado *Cancelada*.
   - Eliminación de una línea de receta de una atención.

   Para evidenciarlo, se debe consultar cada nodo de forma directa (por ejemplo, con `directConnection=true`) y mostrar que todos retornan el mismo resultado.

3. **Alta disponibilidad.** Detener el contenedor del nodo primario y demostrar que el *replica set* elige un nuevo primario, y que el mismo cliente de Python (sin modificar su cadena de conexión) puede seguir ejecutando la consulta (a) e insertando una nueva cita sin que el usuario perciba el problema. Luego, levantar nuevamente el nodo detenido y mostrar que se reincorpora como secundario con los datos escritos durante su ausencia.

4.5. Análisis comparativo: SQL vs. NoSQL

El informe debe contener un análisis teórico que compare las bases de datos relacionales (SQL) con las no relacionales (NoSQL), con una extensión máxima de 4 páginas. El análisis debe apoyarse en la implementación propia del sistema SaludUSM en MySQL (Tarea 2) y en MongoDB (esta tarea), usando ejemplos concretos de ambas implementaciones, y abordar al menos los siguientes aspectos:

- **Modelo de datos y esquema:** tablas, filas y esquema fijo versus colecciones, documentos y esquema flexible. Comparar cómo quedó representada una misma entidad del sistema (por ejemplo, la atención con sus diagnósticos y recetas) en cada modelo.

- **Normalización y redundancia:** normalización a 3FN versus documentos anidados. Indicar qué datos quedaron duplicados en su modelo documental y qué implica actualizarlos (por ejemplo, si cambia el nombre de un médico o de un centro).
- **Integridad y reglas de negocio:** claves primarias y foráneas, eliminación en cascada, *triggers* y restricciones en MySQL, versus lo que ofrece MongoDB. Indicar, para las reglas de negocio del sistema, si en cada motor las garantiza la base de datos o la aplicación.
- **Lenguaje de consulta:** SQL declarativo con `JOIN` versus MQL y el *aggregation pipeline*, comparando una de las consultas de la sección 4.3 en ambos lenguajes.
- **Flexibilidad del esquema:** describir qué cambios requeriría cada motor si SaludUSM decide registrar las alergias del paciente y signos vitales en la atención, cuyos campos varían según la especialidad.
- **Conclusión:** una tabla resumen que compare ambos enfoques según los aspectos anteriores, y una recomendación justificada de qué motor (o combinación de ambos) conviene a SaludUSM para cada módulo del sistema (agendamiento, ficha clínica y panel de gestión).

5. Consideraciones técnicas

Para llevar a cabo esta tarea, se recomienda realizar las siguientes actividades técnicas:

1. Instalar el software de contenedores Docker Desktop y Docker Compose. Usar como referencia https://docs.docker.com/manuals/.
2. Crear el *replica set* de MongoDB en Docker con Docker Compose y subir los servicios.
3. Conectarse al *replica set* desde Python con PyMongo, usando una cadena de conexión como la siguiente:

   ```
   mongodb://mongo1:30001,mongo2:30002,mongo3:30003/?replicaSet=my-replica-set
       &readPreference=primary&appname=MongoDB%20Compass&ssl=false
   ```

   Asegúrese de modificar el archivo *hosts* para que los nombres de los servidores (`mongo1`, `mongo2`, `mongo3`) sean mapeados a la IP local del host. Para consultar un nodo específico puede utilizar una conexión directa, por ejemplo:

   ```
   mongodb://mongo2:30002/?directConnection=true&readPreference=secondaryPreferred
   ```

4. Para leer los datos de la base MySQL de la Tarea 2 desde Python puede utilizar `mysql-connector-python` o `PyMySQL`.
5. Para exportar las colecciones a la carpeta JSON puede utilizar `mongoexport` con la opción `--jsonArray`, o realizar la exportación desde Python.
6. Puede utilizar MongoDB Compass para visualizar los datos, pero todas las operaciones evaluadas deben quedar ejecutadas en el notebook.
7. Documentación acerca de MongoDB, su arquitectura y su lenguaje de consultas en https://www.mongodb.com/docs/manual/.

6. Aclaraciones

1. Es obligatorio utilizar las tecnologías y ambientes solicitados (Docker, MongoDB y Python con PyMongo); existirá descuento en la pauta de no usarse.
2. Cualquier supuesto debe ser especificado de forma explícita en el README; habrá un descuento por cada supuesto no especificado.
3. Obligatoriamente deben existir suficientes datos de prueba para probar cada inciso de la tarea. Pueden reutilizar los datos de la Tarea 2.
4. Secciones o funcionalidades extra no solicitadas no tendrán puntaje adicional.
5. Se recomienda iniciar con tiempo esta tarea.

7. Referencias a material de apoyo

1. MongoDB, Inc. (s.f.). *MongoDB Manual*. https://www.mongodb.com/docs/manual/
2. MongoDB, Inc. (s.f.). *Data Modeling*. https://www.mongodb.com/docs/manual/data-modeling/
3. MongoDB, Inc. (s.f.). *Replication*. https://www.mongodb.com/docs/manual/replication/
4. MongoDB, Inc. (s.f.). *PyMongo Documentation*. https://pymongo.readthedocs.io/
5. Docker, Inc. (s.f.). *Docker Manuals*. https://docs.docker.com/manuals/